

ARD2: full-context anchored row diagrams

DEFINITIONS & PROOFS / ENGLISH EDITION

Definition, 2026-09-14. PDF · NER expander · Python · well-ordering proof · ordinary Lean entry point.

ARD2 is the full-context variant of **Anchored Row Diagrams**. An expression is a finite column diagram, not a path of operations. Each relation group still has just three natural-number coordinates. Unlike ARD, both its row anchor and its root may refer to the current column, and a root need not precede its parent. Software resource guards are not mathematical rules.

1. Finite syntax and root closure

A finite diagram is a sequence of columns $G = (C_0, \dots, C_{m-1})$. A compressed group in column j is

$$(k, p, q), \quad 0 \leq k, q \leq j, \quad 0 \leq p < j.$$

The coordinates are the **row anchor**, **parent column**, and **maximum root**. The child is the containing column j . The group abbreviates all atomic relations (k, t, p, j) with $0 \leq t \leq q$. This is root closure, not row closure. For each pair (k, p) keep only the greatest q ; duplicate relations have no effect. Empty columns remain part of the expression.

The cases $k = j$ and $q = j$ are row SELF and root SELF. They denote the address of this particular child, not a fixed global constant. Both must move with the child during copying. The parent is never SELF. There are no conditions $k \leq p$ or $q \leq p$.

The diagram with no columns is zero, written $\emptyset$. One empty column $[]$ represents one; m empty columns represent the natural number m . The lossless list uses one bracket pair per column, with comma-separated triples:

```
[[[(1,0,1)][(2,1,2)]]
```

This is the three-column seed A_3 . Python and NER store maximum roots. The Lean finite geometry uses explicitly root-closed relations; normalization and duplicate removal are part of the connection between these encodings.

2. Column comparison

Sort each column's compressed groups in decreasing order of (p, k, q) , then compare these lists lexicographically by the same key. Compare diagrams lexicographically from the leftmost column. A proper prefix is smaller at either level. Only the earliest different column and its earliest different group are needed; comparison does not run expansion or search for a path.

Expanding every root and sorting atoms by (p, k, t) gives the same order. For a fixed (p, k) , the greatest differing root already decides the comparison before any later group is reached.

Adjoin a greatest external symbol **Top**, displayed as `Limit` of ARD2. It is not a finite column and cannot be used as a coordinate.

3. Fundamental sequences

The index n is a nonnegative integer. Define $0[n] = 0$; this identity is not a strict descent step. For nonempty G , write $x = m - 1$ and let ∂G delete its last column. Set

$$G[0] = \partial G, \quad G[n] = \partial G \quad \text{for every } n \text{ if } C_x = \emptyset.$$

Otherwise $n > 0$ and the last column is nonempty. Select its greatest group by **row, root, parent**, the key (k, q, p) , and write the group (K, c, r) . This control order is different from the **parent, row, root** comparison order. Put

$$L = x - c > 0, \quad N_b = x + bL, \quad \phi_b(i) = \begin{cases} i, & i < c, \\ i + bL, & i \geq c. \end{cases}$$

The cut is the control parent c . The output has $x + nL$ columns. Take the following union of groups, then normalize:

1. Keep the prefix before c once. For $b = 0, \dots, n$, copy source columns $C_c, \dots, C_{x-1}$, sending (h, p, q) in column j to $(\phi_b(h), \phi_b(p), \phi_b(q))$ in column $\phi_b(j)$.
2. For each $b < n$, put a seam in column N_b . An old last-column group (h, p, q) contributes:
 - $(\phi_b(h), \phi_b(p), \phi_b(q))$ if $h < K$;
 - $(\phi_b(h), \phi_b(p), \min(\phi_b(q), \phi_b(r) - 1))$ if $h = K$, provided the last coordinate is nonnegative;
 - nothing if $h > K$.
3. In the same seam, insert every $(h, \phi_b(c), N_b)$ with $0 \leq h < \phi_b(K)$.

There are $n + 1$ copies of the source block and n seams. The first column of a new copy occupies the preceding seam's position, so their groups are merged. Equivalently one may copy the whole prefix before x in every block: all copies before c coincide.

The new lower-row package has maximum root N_b , **the seam itself**, not $\phi_b(c)$. All four coordinates move in source copies. In particular, a source cut column with row SELF or root SELF rebinds that coordinate to the seam into which it is copied.

After moving a maximum root, take its full root closure. This includes intervening roots skipped by the moving map; merely transporting the old individual roots without reclosing is a different rule. Each call uses finite loops only and never recursively expands its output.

For $H = A_2$, the first terms are

```
H      = [][(1,0,1)]
H[0]   = []
H[1]   = [][(1,0,0),(0,0,1)]
H[2]   = [][(1,0,0),(0,0,1)][(2,1,1),(1,1,2),(0,1,2)]
```

4. Seeds, standard domain and the two well-foundedness statements

The seed A_n consists of n columns, with

$$C_0 = \emptyset, \quad C_j = \{(j, j - 1, j)\} \ (j > 0), \quad \text{Top}[n] = A_n.$$

Thus $A_0 = 0$, $A_1 = []$, and $A_{n+1}[0] = A_n$. Let $D(H)$ contain H and all its finite-expansion descendants, ignoring the $0 \rightarrow 0$ self-loop. Define the finite standard domain and complete notation by

$$U = \bigcup_{n < \omega} D(A_n), \quad U \cup \{\mathbf{Top}\}.$$

Standardness is finite reachability from these seeds, not accessibility or the existence of a semantic representation. A constructor or parser checks structural legality only; it does not certify that arbitrary input belongs to U .

Expansion preserves legality. For every finite G , all columns before its old last column remain unchanged, and $G[n]$ is a complete-column prefix of $G[n + 1]$. That prefix is proper when the old last column is nonempty. Every expansion from a nonzero graph strictly decreases the specified column order.

The paper and ordinary Lean entry point distinguish two conclusions:

- **All legal finite graphs:** the nonzero expansion relation is well-founded. The paper constructs an ordinal-valued rank decreasing on every such step; Lean proves `valid_step_wellFounded`.
- **The standard domain:** the specified column order is a well-order on U , also after adding the greatest external top. Lean proves `standard_strictWellOrder`, `standard_with_top_strictWellOrder`, and the paper-defined-domain version `paper_standard_with_top_strictWellOrder`.

Column order on the entire legal graph space is **not** a well-order. For example,

$$[\emptyset]^{r+1}[(0, 0, 0)] > [\emptyset]^{r+2}[(0, 0, 0)] > \dots$$

is a legal column-order descending chain, but not an expansion chain. The standard-domain restriction in the second conclusion cannot be dropped.

The paper’s axiom bound is $KP_\omega +$ “there exists an uncountable ordinal”, with full Set Induction and without Power Set or Choice. The actual ordinary Lean proof has compiled; its final axiom reports use only `propext`, `Classical.choice`, and `Quot.sound`. This is **not** a Lean encoding of the weak theory’s syntax and an internal derivation in that theory. The definition-fidelity module connects the finite paper rule and its reachable domain to the Lean definitions; it is not a proof about a Python or JavaScript virtual machine. See the repository validation record for verification status.

No inequality between ARD2 and ARD, RPD, Y, wY or IPD is established here. In particular, the name “ARD2,” shared proof methods and larger finite counts do not prove a larger ordinal or an initial-segment relationship. The axiom bound is not claimed to be optimal.

5. Local counts and five displays

Fix the prefix before a column and treat that column as last. Perform a positive-index expansion and discard only the newly appended columns. Repeat until the retained column is empty, then delete it. Its **local count** includes this final deletion, so an empty column has count one. The retained part is independent of the chosen positive index. Rewriting a nonempty last column decreases its local count by exactly one.

In a local step at position j , the source column C_c merges into the retained seam: source row c and source maximum root c both rebind to j . Its parents remain fixed below c . The generated lower-row package has maximum root j . This differs from the old ARD counting algorithm’s assumptions.

For a fixed prefix and position j , there are at most $(j + 2)^{j(j+1)}$ canonical last-column states: for each of $j(j + 1)$ possible (k, p) pairs, choose absence or a maximum root in $0, \dots, j$. Each nonempty local operation strictly decreases this finite column order. Thus the local count terminates without simulating the full repeated [1] trajectory. The seed count sequences $A_1, \dots, A_5$ are the successive prefixes of

1, 5, 55, 969, 23751

These are exact checked values, not a claimed closed formula. Counts are a display, not a lossless input syntax or the comparison key; no inverse from counts is provided.

The file offers exactly five displays, without a duplicated list button:

1. 列表 - the default, lossless one-line triple list.
2. 计数序列 - the exact local counts, using BigInt.
3. 弧线图 - all relation groups, separated by row anchor; each arc connects child to parent and its outlined number is the maximum root. Red anchor dots identify the row's referenced column. Track heights only route the drawing; they are not extra mathematical levels.
4. 邻接表 (文字) - one [] per column, semicolons separating rows from row zero, and commas locating parents from parent zero. Each populated slot contains the maximum root. Interior empty slots are retained, trailing empty slots omitted; the digit 0 is not an empty slot. For example, A_2 becomes [][;1]. This view is reversible.
5. 邻接表 (图) - one complete, compact upper-triangular table per used row, with local counts on a separate line at the top. Shaded diagonal cells contain column indices and serve as both row and column labels. There are no additional outside coordinate labels. A bare row-anchor number appears at the left; off-diagonal cells contain maximum roots. Parent $p < j$ ensures that the omitted lower triangle has no relations, even when a row or root is SELF.

List and count fonts inherit the page's normal font. Graphs are never cropped to a purported complete subgraph. Counting uses an isolated approximately one-second/work budget: if it is exhausted, the display explicitly withholds the counts and still draws all relations when geometry fits. If the geometry itself exceeds its limits, it gives a warning and no partial graph. A resource-limit message is neither an approximate value nor evidence of mathematical nontermination.

6. Software use and checks

Import `ARD2.ne-rewritten.js` through NER's custom-notation control, then select **ARD2**. The registration ID is `ard2-v01`; the default display is 列表. The equivalent-display menu supplies the other four views above. No dependency scripts, network access or persistent cache are required.

Accepted main input includes `A3`, `A3[2][1]`, `Limit[3]`, `Limit of ARD2`, a natural number, and a full triple list. Whitespace is ignored. Natural numbers mean that many empty columns, not a count-sequence encoding. The three NER fundamental-sequence choices `FS`, `FS_alter` and `FS_short` use exactly the same rule with no index offset.

Place the Python file on the module search path and use Python 3.10+:

```
from ard2 import ARD2, ZERO, TOP

a = ARD2.seed(3)
assert str(a) == "[[(1,0,1)][(2,1,2)]"
assert TOP[3] == a
assert a[0] == ARD2(a.columns[:-1])
assert a[1] < a
assert a[1].columns == a[2].columns[:len(a[1].columns)]
assert a.local_step().columns == a[1].columns[:len(a.columns)]
```

ARD2() and ZERO are zero; ARD2.finite(n) has n empty columns; TOP is ARD2(None). The constructor validates coordinates, merges duplicates and normalizes columns. kind is zero, successor or limit; a finite nonzero term is a successor exactly when its last column is empty. fs(n) and [n] are identical. local_step() returns one frozen-prefix local step, not an entire count; it rejects the external top. Natural-number inputs reject negatives, booleans and non-integers.

The Python definition uses only the standard library and arbitrary-precision integers. It intentionally omits parsing, a full counter, drawing, caches and resource guards. Large indices can exhaust time or memory; experiments must impose their own bounds.

The independent Python tests, NER rule tests, and display tests cover full-root closure, the two SELF rebindings, lower-row generation, prefix and comparison behavior, exact small counts, and complete displays. They have explicit time, width and memory checks and do not replace the well-ordering proof. From the repository root:

```
python -B tests/test_ard2.py
node --max-old-space-size=256 tests/ard2_ner.cjs
node --max-old-space-size=256 tests/ard2_display.cjs
```